

Aula 3 — Testes de SPA & PWA

"Como testar PWA offline? Como verificar hydration?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 08/10/2026

Recap da Aula 2

- Playwright avançado (CDP, locators, network mocking)
- storageState, visual regression, sharding
- Trace viewer

Lab Playwright SPA entregue?

Agenda da aula (3h30)

- **90min** — SPA & PWA: hydration, RSC/Next.js, Service Worker, manifest, offline (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: PWA com Lighthouse CI (live coding com replicação pelos alunos)
- **15min** — Wrap-up

Objetivos M3

- **Testar** hydration em SPAs (Next.js App Router, RSC)
- **Testar** Service Worker lifecycle e cache strategies
- **Validar** PWAs (manifest, install prompt, offline mode)
- **Configurar** Lighthouse CI bloqueando merge
- **Aplicar** performance budgets como teste

Particularidades de SPAs

SPAs introduzem fases de execução que testes precisam respeitar:

1. **HTML inicial** (server-side render ou static)
2. **Hydration** — JS anexa event listeners ao HTML
3. **Interativo** — user pode clicar e funciona

| **Teste em fase 1** que parece passar pode falhar em produção quando user clica antes de hydration completar.

Hydration testing

```
test('button works after hydration', async ({ page }) => {  
  await page.goto('/');  
  
  // Aguardar networkidle (heurística: tudo carregado + JS provavelmente hidratou)  
  await page.waitForLoadState('networkidle');  
  
  // Para ser explícito, aguarde marker do framework  
  await page.waitForFunction(() => window.__NEXT_DATA__);  
  
  await page.getByRole('button', { name: 'Click me' }).click();  
  await expect(page.getByText('Clicked!')).toBeVisible();  
});
```

Em React Server Components, hydration acontece **por boundary** — testes precisam aguardar especificamente.

Service Worker testing — desafios

SW roda em **thread separada**, não no main thread. Testes precisam:

1. Forçar registro (não automático em dev)
2. Aguardar `activate`
3. Conhecer cache state
4. Simular network condition

Forçar SW em test env

```
test('SW intercepts API calls', async ({ page, context }) => {  
  await page.goto('/');  
  
  // Aguardar SW estar ready  
  await page.evaluate(async () => {  
    const reg = await navigator.serviceWorker.ready;  
    return reg.active?.state;  
  });  
  
  // Agora SW intercepta requests  
  await page.goto('/dashboard');  
});
```


Cache strategies — testar

```
test('cache-first serves from cache', async ({ page, context }) => {  
  // Primeira visita – popular cache  
  await page.goto('/');  
  await page.waitForLoadState('networkidle');  
  
  // Bloquear network completamente  
  await context.setOffline(true);  
  
  // Recarregar – deve servir do cache  
  await page.reload();  
  await expect(page.getByText('Welcome')).toBeVisible();  
});
```

SW lifecycle — skipWaiting test

```
test('SW updates when new version deployed', async ({ page }) => {  
  // Cenário: nova versão SW disponível  
  await page.goto('/');  
  
  await page.evaluate(() => {  
    return new Promise<void>((resolve) => {  
      navigator.serviceWorker.addEventListener('controllerchange', () => resolve());  
      navigator.serviceWorker.ready.then((reg) => {  
        // Simula update  
        reg.update();  
      });  
    });  
  });  
  
  // Verificar que cliente foi assumido por novo SW  
});
```

Cenários intermediários (waiting state) são onde bugs aparecem.

PWA — manifest validation

```
test('manifest is valid', async ({ page }) => {  
  const response = await page.goto('/manifest.json');  
  const manifest = await response?.json();  
  
  expect(manifest.name).toBeTruthy();  
  expect(manifest.short_name).toBeTruthy();  
  expect(manifest.start_url).toBe('/');  
  expect(manifest.display).toBe('standalone');  
  expect(manifest.icons.length).toBeGreaterThan(0);  
  expect(manifest.icons.find((i) => i.sizes === '192x192')).toBeDefined();  
  expect(manifest.icons.find((i) => i.sizes === '512x512')).toBeDefined();  
});
```

PWA — install prompt simulation

```
test('install prompt fires on supported browsers', async ({ page }) => {  
  let installPromptFired = false;  
  
  await page.exposeFunction('onBeforeInstallPrompt', () => {  
    installPromptFired = true;  
  });  
  
  await page.addInitScript(() => {  
    window.addEventListener('beforeinstallprompt', () => {  
      (window as any).onBeforeInstallPrompt();  
    });  
  });  
  
  await page.goto('/');  
  // Trigger criteria do browser  
  
  expect(installPromptFired).toBe(true);  
});
```

PWA — offline mode test

```
test('offline mode shows cached content', async ({ page, context }) => {  
  await page.goto('/');  
  await page.waitForLoadState('networkidle');  
  
  // Simular perda total de conectividade  
  await context.setOffline(true);  
  
  await page.reload();  
  
  // Conteúdo principal deve estar disponível  
  await expect(page.getByText('Welcome')).toBeVisible();  
  
  // UI deve indicar modo offline  
  await expect(page.getByText('Você está offline')).toBeVisible();  
});
```

Background Sync verification

```
test('queues message for sync when offline', async ({ page, context }) => {  
  await page.goto('/messages');  
  await context.setOffline(true);  
  
  await page.fill('[name=message]', 'Olá!');  
  await page.click('button[type=submit]');  
  
  // Mensagem fica em estado "pending sync"  
  await expect(page.getByText('Pending sync')).toBeVisible();  
  
  // Voltar online  
  await context.setOffline(false);  
  
  // Background Sync deve disparar  
  await page.evaluate(async () => {  
    const reg = await navigator.serviceWorker.ready;  
    return reg.sync.register('flush-outbox');  
  });  
  
  await page.waitForTimeout(2000);  
  await expect(page.getByText('Sent')).toBeVisible();  
});
```

Lighthouse CI — visão geral

```
npm install -g @lhci/cli@0.13.x
```

```
# rodar local  
lhci autorun
```

```
// lighthouserc.json  
{  
  "ci": {  
    "collect": {  
      "url": ["http://localhost:3000/"],  
      "numberOfRuns": 3  
    },  
    "assert": {  
      "assertions": {  
        "categories:performance": ["error", { "minScore": 0.9 }],  
        "categories:pwa": ["error", { "minScore": 0.9 }],  
        "categories:accessibility": ["error", { "minScore": 0.9 }],  
        "first-contentful-paint": ["warn", { "maxNumericValue": 2000 }],  
        "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }]  
      }  
    }  
  }  
}
```

Performance budgets

Conceito de **Tim Kadlec (2013)** popularizado por web.dev.

```
{
  "ci": {
    "assert": {
      "assertions": {
        "resource-summary:script:size": ["error", { "maxNumericValue": 250000 }],
        "resource-summary:image:size": ["error", { "maxNumericValue": 500000 }],
        "resource-summary:stylesheet:size": ["error", { "maxNumericValue": 50000 }]
      }
    }
  }
}
```

PR que aumenta bundle JS além de 250KB → falha. **Death by a thousand cuts** evitado.

Lighthouse CI no GitHub Actions

```
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build
      - run: npm install -g @lhci/cli@0.13.x
      - run: lhci autorun
    env:
      LHCI_GITHUB_APP_TOKEN: ${ secrets.LHCI_GITHUB_APP_TOKEN }
```

Resultado posta comment no PR com link pra report. Falha bloqueia merge.

IA aplicada à análise Lighthouse

Em vez de só ler números, **alimentar relatório pra LLM**:

```
Você é especialista em performance web. Analise este Lighthouse report:
```

```
[JSON do report]
```

```
Identifique:
```

1. Quais regressões são significativas (semanticamente)?
2. Quais são triviais (variação de medição)?
3. Quais ações concretas recomendaria?

```
Responda em formato estruturado.
```

IA faz **interpretação semântica**, não só comparação numérica.

Lab — suíte PWA completa (90min)

Roteiro

1. Clone starter `modulo-03` (PWA exemplo) (5min)
2. Service Worker tests (cache strategies) (20min)
3. Manifest validation (10min)
4. Offline mode tests (15min)
5. Background Sync verification (15min)
6. Lighthouse CI configurado (15min)
7. Budget bloqueando PR (10min)

Pitfalls comuns

- ✗ **Testar SPA sem aguardar hydration** — falha intermitente
- ✗ **Reset de cache Service Worker entre tests** — esquecer = test pollution
- ✗ **Lighthouse CI rodando uma vez só** — variação alta, configure 3+ runs
- ✗ **Budget muito apertado** — bloqueia PRs legítimos por 10ms
- ✗ **Não monitorar Web Vitals em produção** — divergência entre lab e prod

Atividade — Lab PWA Testing (10 pts)

Entrega: ver Canvas.

Entregar repo com:

1. **SW lifecycle testado** (install/activate/fetch interceptados) — 3pts
2. **Offline mode test** funcionando — 3pts
3. **Lighthouse CI com 3 budgets** bloqueando merge — 3pts
4. **Manifest + install prompt** simulados — 1pt

Leituras obrigatórias

- Archibald, J. — *Service Workers Testing Strategies* (web.dev)
- web.dev — *Lighthouse CI* (oficial)
- Lock, A. — *Testing Service Workers* (artigo)
- **Mobile.dev Docs** — preparação M4

Próximo módulo (M4)

Automação Mobile com Maestro

- YAML declarativo
- Conditional flows + runFlow
- Maestro Studio
- Comparativo Maestro vs Appium vs Detox
- Cloud devices

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith